Assignment 3: Recommender Systems

Name: Sushantak Parashar Jha
Roll No: M25CSA035
Program: M.Tech in AI
Course: Machine Learning With Big Data(CSL7110)
GitHub Repository and Pages Link: Link

1. Introduction

Recommender systems are used a lot in modern apps like e-commerce sites, streaming services, and social media sites to give users personalized recommendations. These systems look at how users act, what items are like, and how they interact with each other to guess what items they might like.

The goal of this assignment is to use the MovieLens dataset to put into practice and study different kinds of recommendation methods. The implemented methods encompass conventional recommendation strategies, such as content-based filtering and collaborative filtering, alongside more sophisticated techniques, including matrix factorization, neural network-based recommendation, reinforcement learning methodologies, and explainability frameworks.

We put each method into action and tested it to see what works and what doesn't. Also, methods for making recommendation systems easier to understand and more transparent are looked into.

---

2. Dataset Description

The dataset used in this assignment is the **MovieLens dataset**, which is widely used for evaluating recommendation algorithms. The dataset contains information about movies and ratings provided by users.

The following files from the dataset are used:

- **movies.csv** – Contains movie titles and genre information.
- **ratings.csv** – Contains user ratings for movies.
- **links.csv** – Contains links to external movie databases.
- **tags.csv** – Contains user-generated tags for movies.

The dataset contains:

- Multiple users
- Thousands of movies
- User ratings ranging from **0.5 to 5.0**

These ratings are used as the primary signal to learn user preferences and generate recommendations.

---

3. Content-Based Filtering

Content-based filtering suggests things that are like things the user has liked in the past. This method uses movie features to figure out how similar two movies are.

This implementation uses movie genres as the main way to describe each movie. The TF-IDF (Term Frequency–Inverse Document Frequency) method changes the genres into numerical vectors. This change turns information about the genre of text into numerical feature vectors.

After the TF-IDF vectors are made, cosine similarity is used to find movies that are similar to each other. The user is shown movies that are most similar to the ones they have already seen.

This method works well when the features of the items are useful and easy to find. But it might have trouble when user preferences are complicated or when there isn't much metadata.

```
recommend_movies("Toy Story (1995)")
```

	title	similarity_score
2355	Toy Story 2 (1999)	0.880446
7355	Toy Story 3 (2010)	0.821047
3595	Toy, The (1982)	0.538018
2539	We're Back! A Dinosaur's Story (1993)	0.456423
26	Now and Then (1995)	0.421651

Content-based recommendation output

---

4. Collaborative Filtering

Collaborative filtering suggests things to buy based on what other people with similar tastes have liked. This method uses patterns in how users behave instead of item features.

To start, a matrix of user-movie ratings is made, with users in the rows and movies in the columns. The rating a user gave a movie is in each cell.

After that, cosine similarity is used to find out how similar users are by comparing their rating vectors. People who rate things in the same way are called neighbors.

Recommendations are made by guessing ratings based on ratings from users who are similar to you.

Collaborative filtering can find patterns in how users act, but it has problems like:

- Cold start problem
- Data sparsity

```
recommend_cf(5)
```

	title	predicted_rating
10	American President, The (1995)	5.000000
13	Nixon (1995)	5.000000
24	Leaving Las Vegas (1995)	5.000000
202	French Kiss (1995)	4.654903
253	Outbreak (1995)	4.406271
314	Forrest Gump (1994)	4.200561
334	Speed (1994)	4.046790
505	Ghost (1990)	4.003216
510	Silence of the Lambs, The (1991)	4.000000
615	Independence Day (a.k.a. ID4) (1996)	4.000000

Collaborative filtering recommendation output

---

5. Matrix Factorization (SVD)

Matrix factorization techniques aim to discover hidden patterns in user–item interactions. One of the most common methods is **Singular Value Decomposition (SVD)**.

In this approach, the user–movie rating matrix is decomposed into three matrices:

- User latent feature matrix
- Singular value matrix
- Movie latent feature matrix

These matrices capture hidden relationships between users and movies. By multiplying the matrices together, predicted ratings can be generated.

Matrix factorization techniques are widely used in real-world recommender systems because they can capture complex relationships in sparse datasets.

```
recommend_svd(1)
```

	title	predicted_rating
659	Godfather, The (1972)	4.009045
793	Die Hard (1988)	3.303897
922	Godfather: Part II, The (1974)	3.303409
958	Stand by Me (1986)	2.869847
1067	Jaws (1975)	2.859630
1445	Breakfast Club, The (1985)	2.785549
1544	Lady and the Tramp (1955)	2.603560
1545	Little Mermaid, The (1989)	2.455787
2110	Christmas Story, A (1983)	2.403890
2996	Snatch (2000)	2.372148

SVD recommendation output

---

6. Hybrid Recommendation Model

Hybrid recommender systems combine multiple recommendation techniques to improve performance. In this assignment, a hybrid model combines **content-based filtering** and **collaborative filtering**.

The final recommendation score is calculated using a weighted combination:

Final Score = α × Content-Based Score + (1 − α) × Collaborative Score

This approach helps overcome the limitations of individual models. For example:

- Content-based methods handle cold-start problems better.

- Collaborative filtering captures community preferences.

By combining both methods, the hybrid model generates more robust recommendations.

```
recommend_hybrid(24)
```

	title	hybrid_score
43	Seven (a.k.a. Se7en) (1995)	1.312699
224	Star Wars: Episode IV - A New Hope (1977)	1.130531
898	Star Wars: Episode V - The Empire Strikes Back...	0.941360
911	Star Wars: Episode VI - Return of the Jedi (1983)	0.938703
969	Back to the Future (1985)	0.903268
2145	American Beauty (1999)	0.883946
2226	Fight Club (1999)	0.852452
6315	Departed, The (2006)	0.842689
6330	Children of Men (2006)	0.830622
6772	WALL·E (2008)	0.816302

Hybrid recommendation output

---

7. Neural Network Recommender

This part shows how to use user and movie embeddings to make a recommender system based on a neural network.

Embeddings enable the neural network to acquire latent representations of users and films. These embeddings show complicated relationships that might not be easy to show with more common methods.

The neural network takes in the user's and the movie's IDs and guesses what rating the user would give the movie.

The model is trained using:

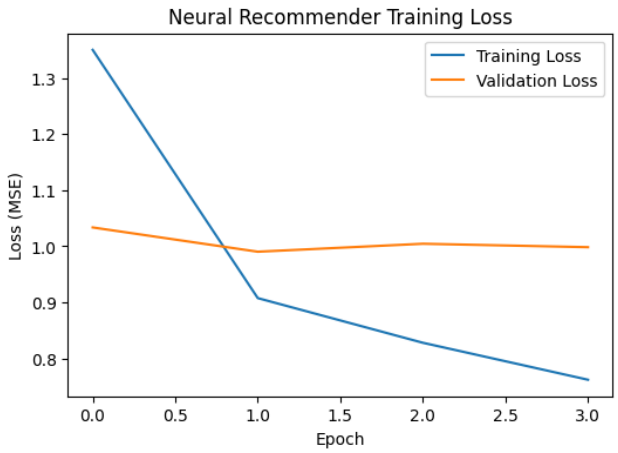
- Mean Squared Error (MSE) loss
- Adam optimizer

To reduce overfitting, the following techniques were applied:

- Dropout layers

- Early stopping

Training Loss Visualization



Neural network training loss plot

The training loss decreases steadily across epochs, indicating that the model is learning useful patterns from the data. The validation loss remains relatively stable, suggesting that the model generalizes reasonably well to unseen data.

---

8. Reinforcement Learning Recommender

Reinforcement learning can be utilized in recommender systems by conceptualizing recommendations as a sequential decision-making challenge.

This assignment uses a Multi-Armed Bandit method. The system has to choose which movie to recommend, and each movie is like a arm.

The algorithm balances two objectives:

- **Exploration** – trying new movies to gather information
- **Exploitation** – recommending movies known to perform well

Rewards are simulated based on user ratings. Movies with higher ratings produce higher rewards, encouraging the system to recommend similar movies in the future.

This approach demonstrates how recommender systems can adapt dynamically based on user feedback.

9. Explainability in Recommender Systems

Modern recommender systems often operate as black-box models, making it difficult to understand why certain recommendations are made.

To improve transparency, several explainability methods are implemented:

Feature-Based Explanation

Recommendations are explained by highlighting shared movie genres between recommended movies and movies previously liked by the user.

Neighborhood-Based Explanation

Recommendations are explained by identifying similar users whose ratings influenced the prediction.

Model-Agnostic Explanation (SHAP)

SHAP (SHapley Additive Explanations) is used to interpret predictions from the neural recommender model.

---

10. Evaluation of Explainability

Methods for explainability make it easier to understand recommendation systems.

Users find feature-based explanations the easiest to understand because they directly refer to things that can be seen, like movie genres.

Neighborhood-based explanations give social context by showing that other people who are like you also liked the suggested items.

SHAP and other model-agnostic methods can help us understand complicated machine learning models, but the explanations may be hard to understand for non-technical people.

Explanations can also help find biases in recommendation systems. For instance, a recommender system might only recommend popular movies or certain types of movies. Finding these kinds of patterns can help developers make recommendations that are more fair and diverse.

11. Assumptions

The following assumptions were made during the implementation:

1. Movie genres were used as the primary features for content-based filtering.
2. Ratings greater than or equal to 4 were considered positive feedback for reinforcement learning rewards.
3. User and movie identifiers were used as inputs for the neural network model instead of additional metadata features.
4. The hybrid recommender used a fixed weighting between content-based and collaborative filtering scores.

12. Conclusion

We used the MovieLens dataset to look at a few different ways to make recommendations for this assignment.

Content-based filtering made recommendations that made sense based on movie traits, while collaborative filtering used patterns in how users acted to make personalized suggestions. Matrix factorization methods found hidden connections between users and movies, which made predictions more accurate.

Recommenders that use neural networks showed that they could learn complicated interactions by embedding representations. Reinforcement learning methods showed how recommendation strategies can change based on what users say.

Lastly, techniques for making things easier to understand were used to make the recommendation outputs clearer. These methods help users trust the system more and give developers a better idea of how recommendation systems work.